MiDoctor HANDBOOK

Patient app, provider app and operator console — what they do and how to run them.

MiDoctor

An Indian telemedicine product: patients find a doctor, book a slot, hold the consultation over video, and keep the prescription and records that come out of it. Doctors work the other side of the same screens. Supervisors and support staff work in a separate web console.

Three clients, two binaries. This handbook covers what each one does, how to run it, the rules it enforces, and the parts that are honestly not finished.

---

01 · PLATFORM

What this is

The repository root is a Flutter app. `functions/` is the MiDoctor API — a TypeScript Express app on Cloud Functions, plus a Twilio carrier check, a storage trigger and three scheduled sweeps.

AUDIENCES	BINARIES	REPOSITORIES
3	2	18
Patient, provider, operator	`main.dart` and `main_admin.dart`	Each one a backend surface

TESTS	
385	
Flutter, plus 83 on the API	

Two binaries, and why

Patient and provider share `lib/main.dart`. Which one you get is decided by your *server session*, not by a setting — and the two live in separate `StatefulShellRoute` trees, so it is structurally impossible to render a provider tab inside the patient shell.

Supervisors, admins and support staff use the operator console at `lib/main_admin.dart`. It is a separate entry point for three reasons worth keeping:

- Nothing under `lib/admin/` is reachable from `lib/main.dart`, so approval logic and reviewer copy are never compiled into the APK that ships to the doctors being reviewed. A CI job fails if that stops being true.
- Reviewing a degree certificate on a phone is not a thing anyone should do. Modelling it as "another tab" invites exactly that.

- It can sit behind a VPN or an IdP without any of that touching the app.

Identity versus authority

Firebase Auth proves **who you are** — Google, Apple, or an India phone OTP. There is no email and password. The Firebase ID token is then exchanged for a **MiDoctor session**, which is what proves **what you may do**. They are separate objects, and the app never authorizes off Firebase.

> Everything runs against in-memory sample data by default (`USE_FIXTURES=true`). Setting it to `false` swaps all eighteen repositories to their API-backed versions in one reviewable file, `lib/core/feature_providers.dart`.

02 · PLATFORM

Running it

Dart SDK `>=3.3.0 <4.0.0`. The API uses **pnpm**, not npm — `npm install` fails on pnpm's `node_modules` layout.

The app

```
flutter pub get
flutter run --dart-define=USE_FIXTURES=true     # sample data, no backend
flutter run -d chrome                            # web, same flags
```

The operator console

```
flutter run -d chrome -t lib/main_admin.dart
flutter build web -t lib/main_admin.dart --output build/admin
```

Against a real backend

```
cd functions && pnpm install
firebase emulators:start --only functions,firestore

flutter run \
  --dart-define=USE_FIXTURES=false \
  --dart-define=API_BASE_URL=http://10.0.2.2:5001/<project>/asia-
south1/api
```

The Firestore emulator needs **JDK 21 or newer**; the functions emulator alone does not. Secrets go in a gitignored `functions/.secret.local`.

Build-time configuration

KEY	VALUES	DEFAULT
ENV	`dev · staging · prod`	`dev`
API_BASE_URL	any URL	`http://10.0.2.2:8080`
USE_FIXTURES	`true · false`	`true`

Credentials you will need to supply

`lib/firebase_options.dart` and `android/app/google-services.json` are gitignored. Regenerate them with `flutterfire configure`, or copy the credential-free stubs from `tool/ci/` for a build that compiles without a Firebase project.

> **Google Sign-In on web**
>
> It reads neither `firebase_options.dart` nor `google-services.json`. It needs the Web OAuth client id in a `<meta name="google-signin-client_id">` tag in `web/index.html`, where it sits commented out with instructions. Without it, sign-in fails at runtime with no useful error.

Checks and builds

```
flutter analyze --fatal-infos          # must be clean; CI enforces
dart format --set-exit-if-changed lib test
flutter test                           # 385 tests
flutter test --tags golden             # excluded from CI (host fonts)
flutter build appbundle                # ship the AAB, never the universal
APK
cd functions && pnpm test              # 83 tests, no emulator needed
```

03 · GETTING STARTED

A first walkthrough

Sample data has no identity provider behind it, so both apps offer an explicit sample sign-in. Without it the mock data was unreachable — the app opened on a sign-in screen it could not get past.

1. **Start the app and pick a persona**

   On the sign-in screen, **Explore with sample data** signs you in as Priya Sharma, a patient who already has appointments, records and prescriptions. Below the divider, **Explore as an approved doctor** and **Explore as a doctor awaiting verification** reach the other half of the binary — otherwise unreachable, because signing up as a doctor needs Firebase and the approval that opens the provider shell is an operator decision taken in a different process.

2. **Book something, and watch it land**

   Home → find a doctor → pick a slot. The booking appears in **Appointments**, and that slot stops being offered to anyone else. Cancel it and the slot returns to the pool.

3. **Upload a record**

   Records → upload. It is unreadable for about three seconds while it "scans", then clears. The quarantine state is real, not skipped — that is what the server does with an uploaded file before anything may open it.

4. **Grant a doctor access, then take it back**

   Sharing → grant. The doctor's view opens to exactly what the grant covers. Revoke it and the view closes. Every grant has a mandatory finite expiry; there is no "until I revoke".

5. **Cross the shell**

   Sign out, come back as the approved doctor, and issue a prescription from a completed consultation. Sign back in as the patient: it is in their list, and the appointment stops saying "none".

6 **Open the console**

`flutter run -d chrome -t lib/main_admin.dart`, then **Open with sample data**. Submit credentials as the pending doctor and the application shows up in the review queue. Rate a consultation as the patient and it appears in the moderation queue.

> **Why the sample data behaves like this**
>
> Every fixture reads and writes **one** in-memory store, `FixtureBackend`. They used to be independent, and the product did not work even though every screen demoed correctly: booking returned an appointment the appointments list had never heard of, an uploaded record stayed "checking…" forever, and a prescription written by a doctor never reached the patient.
>
> It also enforces the real rules rather than accepting anything — slot exclusivity by the same deterministic id the server uses, the 180-day consent ceiling, the 14-day rating window, the MoHFW drug lists. A fixture that accepted what the server rejects would teach the UI a rule that does not hold.

What sample data will not pretend to do

In the console, opening a credential document, suspending an account and assigning a role all refuse with a clear message. There is no file store behind the fixture, and a console that reports "account suspended" when nothing was suspended is the most dangerous kind of mock.

04 · PATIENT APP

The patient app

Four tabs — Home, Appointments, Records, Profile — with everything else pushed over them.

Signing in

Three methods, all of which prove identity only: **phone OTP** (India, with a carrier and VoIP check first, so an internet calling number is rejected before an OTP is spent on it), **Google**, and **Apple** on iOS and macOS. Whatever succeeds, the Firebase ID token is exchanged for a MiDoctor session.

A new patient then completes onboarding — name, date of birth, blood group, allergies, conditions, emergency contact — before the shell opens.

Home

The next appointment as a hero card with a live countdown, and a grid of quick actions: **Records**, **Prescriptions**, **My medicines**, **Who can see my records**, **Support**.

Finding and booking a doctor

Search with a debounced field and a filter sheet — specialty, city, consultation mode, fee, rating. The detail page carries qualifications, the NMC registration number, languages, fee and published ratings.

Booking is a date strip plus a slot grid. Picking a slot takes a **hold** with a visible countdown; the hold lapses and the slot returns to the pool if you do not finish. If nothing is free, you can join a **waitlist** — and when a slot opens, the notification says so on a first-come, first-served basis.

Appointments

Upcoming and past. The detail screen offers cancel, reschedule, check-in, join, the prescription if one was issued, the doctor's consultation note, and rating once it is over.

Rescheduling is one call, never cancel-then-book. The server takes the new slot lock and releases the old one in a single transaction, so the worst outcome is that you keep your original time. The reference code survives — a moved appointment is the same appointment.

Where a queue exists, the appointment shows your **position** — derived from counts the server returns, never from anyone else's appointment details.

The consultation

Four stages on one screen: a **consent gate**, a waiting room, the live call, and chat. If the connection degrades you can drop to audio without leaving.

The consent gate renders its text from `TelemedicineConsent`, and the exact wording and version are sent to the server, which stores a SHA-256 of it. That is why the copy must not be machine-translated and why changing the wording means bumping the version — a hash of text nobody saw is not evidence of anything.

> **Video does not work on web.** The 100ms SDK ships Android and iOS only. On web the app binds an unsupported provider that fails as an ordinary error telling you to join from your phone, rather than throwing a plugin exception mid-call.

Prescriptions and medicines

Prescriptions list and detail, with the PDF — fetched from the server's write-once copy, falling back to a locally rendered one when offline, which says so. A prescription is a legal document: the doctor's details are snapshots taken at issue time, so it renders exactly as issued even after they edit their profile.

You can request a **repeat**; the doctor must give a real reason to decline, from a closed set plus a mandatory note. If a follow-up date was set, it is shown above the repeat button — somebody due a review should be booking one, not reordering the same course.

My medicines turns those prescriptions into a day: what to take this morning, this afternoon, this evening, tonight. Tick a dose off, or skip it, or take the mark back. Adherence appears on the prescription as a count — "11 of 14 doses ticked off" — never a percentage, because a percentage borrows the authority of a measurement nobody took.

Records and sharing

Upload from camera, gallery or a document picker. The file is unreadable until it has been scanned. Sharing has three tabs — active grants, pending requests from doctors, and the access log. **The log records denials too.**

Notifications

A notification centre plus per-kind toggles and quiet hours (22:00–07:00 by default, wrapping past midnight). Two kinds are **mandatory** and ignore both the toggle and quiet hours: an appointment that changed, and an account update. A patient must not be able to opt out of the only warning that their consultation is not happening.

Signed-in devices

Identity is Google, Apple or an OTP, so there is no password to change. Settings lists every live session, the device in your hand first, and ends any of them — or all the others at once.

It shows platform and last-seen and **no location**. A "signed in from Mumbai" line reads as reassuring security detail and is really a location history shown to whoever is holding an unlocked phone, including the person a patient might be hiding a consultation from. Revoking bumps `permissionVersion`, so a stolen access token dies on its next request rather than fifteen minutes later.

Profile, settings, privacy

Edit profile, notification settings, language, and a privacy screen carrying the DPDP rights summary, data export, and account deletion. The emergency contact you gave at onboarding appears on the profile tab with a call button — it opens the dialer with the number typed in and does not place the call.

05 · PROVIDER APP

The provider app

Four tabs — Today, Schedule, Patients, Profile — but only for an **approved** doctor. Every other verification status is pinned to the verification screen.

Getting approved

The checklist wants all documents present, a registration number, and MFA enrolled before it will let you submit for review. **A rejected document counts as not provided.** Identity is DigiLocker only — there is deliberately no Aadhaar credential kind.

MFA is TOTP. The seed and the recovery codes expire off the clipboard after 60 seconds, and a later copy by the user is left alone.

Today

The day's list, with check-in and join, and whatever needs attention: new consent requests, refill requests waiting on a decision.

Schedule

Weekly availability rules, plus exceptions. Block a day and that day's slot grid genuinely empties for every patient.

> Availability is anchored to **IST (+05:30)**, never to server or caller local time. Cloud Functions runs in UTC, so building slots from a local date shifts every doctor's hours by five and a half hours in production while looking correct on an Indian dev machine. India has one timezone and no DST, so a fixed offset is exact.

Patients

Only patients who have granted access, and only what the grant covers. Records opened here are logged against the grant.

Prescribing

Drug search from the catalogue, dose, frequency, duration, instructions, diagnosis, advice, follow-up date. Drugs you may not prescribe are shown **disabled with a reason**, not hidden — the MoHFW telemedicine drug lists are a legal constraint, and the reason is the part a doctor needs.

The client's copy of the table only decides what to grey out. The server re-resolves every item from the catalogue by id and derives follow-up status from appointment history, so a repackaged client that sends the request anyway is refused.

Saved prescribing sets

Doctors prescribe the same twenty medicines. Save a set from the composer, apply it in one tap.

A set grants nothing. Applying fills the composer; issuing still runs every item through the drug lists. A set saved on a follow-up may hold a List B medicine, so applying filters to what is prescribable on *this* consultation, and the picker says how many items will be left behind before you commit. Only drug ids

and doses are stored — names and classifications are read back from the catalogue every time, so a drug moved to List B starts refusing in every set containing it that same day.

Sets are private to the doctor who saved them. No sharing, no practice library: one clinician's set becoming another's default is how a prescribing habit spreads without anybody deciding it should.

Patient timeline

One patient's history with you in a single column — consultations, notes, prescriptions, and the records they shared. Composed from what you can already read rather than from a "give me everything about this patient" endpoint, which would be one authorization mistake away from being a patient-history API.

It separates **your own clinical acts** from **the patient's records**. A note you wrote does not stop being yours when a grant lapses; a scan they uploaded is visible only while consent covers it. A lapsed grant is said in a banner, never inferred from an empty list.

Consultation notes

Written once per appointment, once the consultation has begun. **Append-only**: corrected by addendum, never edited or deleted.

A clinical note is evidence of what a clinician thought at a point in time, and the occasions one most needs changing are exactly the ones where somebody has an interest in the earlier version disappearing. The author, their registration number and the timestamp all come from the session and the clock — every part of the signature a client could supply is a part it could forge.

Ratings

Your published ratings, and a reply. Ratings are moderated before they publish.

06 · OPERATOR CONSOLE

The operator console

A web app for supervisors, admins and support staff. It reuses `lib/core` entirely — the same HTTP client, auth interceptor, single-flight refresh and error mapping — because the authorization story is identical and having two of those is how they drift. Only the shell, routing and screens differ.

SCREEN	SCOPE	WHAT IT DOES
Verification queue → applicant	`provider: review` `provider: approve`	Read each credential document in-page, accept or reject it with a structured reason, then approve — which composes the public directory listing — or reject the application.

SCREEN	SCOPE	WHAT IT DOES
Accounts	`user:suspend` `user:set_role`	Look up by id, suspend or deactivate with a recorded reason, reactivate, assign a role.
Ratings	`provider:review`	Publish, hide or remove. This queue is a direct lever on which doctors search surfaces.
Support	`support:ticket_read`	Queue, thread, reply, escalate, close.
Overview	per queue	What is waiting, with how long the oldest item has waited. Sections are **omitted, not zeroed**, when the scope is missing — a count that reaches the client and is merely hidden has already left the building.
Access log	`user:suspend`	Every recorded read of one account's records, **refusals included** — a doctor repeatedly trying records they hold no grant for is the pattern a log exists to surface. Reading it is itself recorded.

Two deliberate absences

There is **no user search**. An operator acting on an account already has its id, and a search box hands a helpdesk the ability to enumerate patients by name.

There is **no patient-context panel** on a support ticket. Support staff hold no consent grant, and a "recent appointments" sidebar is the fastest way to turn a helpdesk into an unlogged route into someone's medical history.

Notes

- The first admin is created with `pnpm grant-role`. Nothing in the API can mint one.
- Deployed as its own Firebase Hosting target, separate from the app's, with `X-Frame-Options: DENY` and a no-referrer policy.
- English only, deliberately. It is internal staff tooling, and translating a review queue nobody outside the company sees buys nothing.
- `resolveAdminRedirect` is pure and unit-tested exactly like the mobile one, and deliberately *not* shared with it — one function serving both would mean a single edit could let a patient into the review queue.

07 · PLATFORM

Rules it enforces

These live in the domain layer and are tested directly. They are not UI conventions, and re-implementing them in a screen is how they drift.

AREA	RULE
Drug lists	MoHFW telemedicine lists. OTC and List A are prescribable on a first consult; **List B is refill-only** and blocked unless it is a follow-up; prohibited never. Enforced server-side; the client copy only greys things out.
Consent	Every grant has a mandatory finite expiry, ceiling 180 days. There is no "until I revoke". Revoke is one tap. The access log records denials.
Cancellation	Only while upcoming. Free-cancellation window 24h — a placeholder with no business authority behind it.
Join window	Opens 15 minutes before the start, closes 30 minutes after the scheduled end. In-person is never joinable.
Ratings	One per completed appointment, editable for 14 days, moderated before publish.
Consultation notes	Append-only. One per appointment, only once the consultation has begun, corrected by addendum.
Rescheduling	One call, never cancel-then-book. Status stays confirmed; the reference code survives.
Notifications	A closed set of kinds. Each decides which toggle silences it, where it leads, and whether it may arrive at 3am. Appointment changes and account updates are mandatory and bypass quiet hours.
Verification gate	All documents, plus registration number, plus MFA, before submission. A rejected document counts as not provided.
Dosing	The parser reads `1-0-1`, the Latin abbreviations and a few English phrases. Everything else gets **no schedule at all**.
Prescribing sets	A set grants nothing. Stored as drug ids and doses; classification resolved from the catalogue on every read.
Audit	The access log records denials, and reading it as an operator is itself recorded.
Dose log	Self-report. Never shown to a doctor. A dose that is not due yet cannot be ticked; the whole of today can.

> **Why the dosing parser refuses**
>
> `frequency` is free text a doctor typed. Guessing at the rest would be an app telling somebody to take a drug at a frequency nobody wrote down, with the prescription on screen as apparent authority for it.
>
> Concretely: "Once weekly" contains "once", so a phrase table consulted first schedules 60,000 IU of vitamin D *every morning* instead of every Sunday. Non-daily intervals are therefore refused before the phrase table runs. An unparsed frequency renders as the doctor's own words with no reminders — which is exactly what a paper prescription does.

Notification content

Neither the title nor the body may carry clinical content. They render on a lock screen, mirror to a paired watch, and are read by whoever is holding the phone. "Prescription ready" is fine; naming the drug is a disclosure with no consent record and no way to withdraw it. The detail lives behind the tap, inside the app, behind authentication.

How it is built

CONCERN	CHOICE
State	`flutter_riverpod` ^3.4 — no codegen; `riverpod_generator` is unusable on this Flutter version, so providers are written by hand
Routing	`go_router` ^17.5, hand-written paths
HTTP	`dio` ^5.11 behind an `ApiClient` wrapper
Identity	Firebase Auth — Google, Apple, India phone OTP
Authorization	MiDoctor server session, its own JWT
Secure storage	`flutter_secure_storage` ^11 — iOS Keychain, Android KeyStore
Telehealth	`hmssdk_flutter` (100ms) behind a `TelehealthProvider` seam
Localisation	ARB — ~490 keys in English, ~474 in Hindi

Three layers per feature

```
lib/features/<feature>/
  domain/        immutable models, enums, business rules. No Flutter, no
Dio.
  data/          abstract Repository + Fixture… + Api…
  presentation/  providers/controllers + ConsumerWidget screens
```

Nothing above `ApiClient` knows about Dio or HTTP status codes. Errors are always a `Failure` with a coarse kind and a machine code. Screens are views: mutations go through a controller or a top-level function beside the provider, which invalidates what it affected.

Session and roles

The session carries the user id, session id, access token, role, account status, provider status, scopes and a permission version. The **access token is memory-only** and never written to disk. The **refresh token lives in secure storage only** and is rotated on every refresh. The device id is a random per-install string, deliberately not a hardware id (DPDP Act).

Redirect precedence

The order matters, and `resolveRedirect` is a pure function covered by 21 unit tests:

1. Session still loading → splash
2. No session → auth
3. Account not usable → blocked

4. Role has no mobile UI → blocked
5. Patient without onboarding → onboarding
6. Provider not approved → verification
7. Cross-shell containment — a patient can never land on a provider route, or the reverse

Network

- `ProblemJson` parses RFC 9457 `application/problem+json` into a `Failure`, honouring `Retry-After` and `x-request-id`.
- `AuthInterceptor` injects the bearer token, and on a 401 *or* a body carrying `TOKEN_STALE` refreshes once and replays through a bare Dio that has no interceptors and therefore cannot recurse.
- `RefreshCoordinator` makes refresh single-flight. With rotating refresh tokens and reuse detection, a parallel second refresh looks like theft and revokes the whole session family.

Design system

Three files under `lib/core/theme/`, and nothing outside them picks a colour, a radius or a duration. Both colour schemes are written out rather than seeded — seeding from a teal brand hue tints every grey green and drags clinical content toward the product colour.

Status colour comes from `AppTones`, not from primary and error. A completed appointment is not an error and a scanning record is not a success. A tone ships a contrast-checked foreground *and* container for both brightnesses, where a raw colour at 12% alpha is only ever checked in whichever mode the author had open.

Motion is one language: 120ms for press, 180ms for state swaps, 240ms for pages, 360ms for first-run reveals; only opacity and transform are animated; every animation checks the OS reduce-motion setting. Loading is a skeleton, held back 160ms so a cached list does not flash grey bars for 60ms.

> **Two traps worth knowing**
>
> **A stateful list item in a builder needs a key.** Entrance animations are stateful, so an unkeyed one is matched positionally — switch a filter and row three's finished animation controller is handed to a different record, which arrives already faded in. Key by domain id.
>
> **Fixed heights must come from `MediaQuery.textScalerOf`.** A hard-coded tile height is a guaranteed overflow the moment someone turns the system font up, and this audience turns it up. Widget tests render the main screens at 1.3× and 2× and fail on any overflow.

Offline

Appointments and prescriptions are cached through `ClinicalCache`, as a decorator so the policy lives in one place and wraps the fixture too. Every clause matters: encrypted at rest, **never on web**, **metadata only** — never record bytes or a prescription PDF — a **3-day expiry** with anything already past

dropped on the way out, wiped on sign-out, and **the screen says so**, escalating its wording past 24 hours because one phrasing cannot carry both "an hour old" and "two days old".

That last clause justifies the rest. Silently rendering yesterday's appointments as today's is a wrong answer delivered confidently, which for a list somebody plans their day around is worse than an error. A non-network failure is never masked by the cache: a 403 is an answer, and serving last week's list would hide a suspension.

09 · ENGINEERING

The API

An Express app on Cloud Functions in `asia-south1`, plus `verifyIndianCarrier` (a Twilio callable), `inspectUpload` (a storage trigger) and three scheduled sweeps. Wire values are `UPPER_SNAKE` throughout.

Invariants the client depends on

- **Slot lock ids are deterministic** — `<doctorId>__<startMillis>`. That is the entire no-double-booking guarantee: two phones addressing the same document id lets a Firestore transaction serialise them, the way a Postgres exclusion constraint would. Random ids would silently remove it.
- **Refresh tokens are single-use and stored hashed; reuse revokes the family.** This is why single-flight refresh exists on the client.
- **A permission-version mismatch returns** `TOKEN_STALE`, which the interceptor answers by refreshing and replaying. Every authorization change increments it, so a suspension lands on the next request rather than up to fifteen minutes later.
- **Uploaded bytes never pass through the API.** The client PUTs to a signed URL, the object lands in a `quarantine/` prefix nothing can read, and the trigger promotes it only after confirming the type from magic bytes and stripping EXIF.
- **Firestore rules deny everything, deliberately.** The app never holds a Firestore credential; the API uses the Admin SDK, which bypasses rules. Authorization has exactly one implementation — a second, weaker implementation of consent expiry is worse than none.

> **New queries usually need a new index**
>
> An equality filter plus an `orderBy` or a range on a *different* field needs an entry in `firestore.indexes.json`. Equality-only conjunctions do not. Nothing fails locally, the emulator serves anything, and nothing fails at build — it fails as `FAILED_PRECONDITION` the first time a real user opens the screen. Five accumulated before an audit caught them.

Route table (abridged)

METHOD	PATH	REQUIRES
POST	/v1/auth/session	a Firebase ID token
POST	/v1/auth/refresh	a refresh token
GET	/v1/doctors, /:id	authenticated
GET/POST	/v1/booking/slots, /hold, /book	appointment:create
POST	/v1/appointments/:id/reschedule	appointment:create
GET/POST	/v1/appointments/:id/note	participant · prescription:write
POST	/v1/notes/:id/addendum	prescription:write
GET/POST	/v1/records	records:read_own
GET/POST	/v1/consent/grants, /requests, /log	consent:*
POST	/v1/prescriptions	prescription:write
GET	/v1/prescriptions/:id/pdf	patient or issuing doctor
GET	/v1/rx/:code	**none** — a pharmacist holds no token
GET	/v1/medications/doses?from=	prescription:read_own
PUT/DELETE	/v1/medications/doses/:id	prescription:read_own
GET/PUT	/v1/notifications/preferences	authenticated
POST	/v1/credentials/submit	credentials:submit
GET/POST	/v1/review/providers/:userId	provider:review

The full table is in `functions/README.md`.

> **A trap this API has already fallen into**
>
> Dose ids contain `#`. Interpolated raw into a URL path, `#` starts the fragment — and a fragment is never sent, so every request silently addressed the prescription id alone. Percent-encode any id that carries one.

10 · ENGINEERING

Security & privacy

Jurisdiction, for the record. HIPAA is US law. This product is India-facing, where the **DPDP Act 2023**, the **Telemedicine Practice Guidelines 2020** and the SPDI Rules bind. §164.312 is the stricter checklist, so building to it satisfies DPDP too. Every HIPAA §164.312 technical safeguard a mobile client can implement is implemented; what remains is server-side or organizational. The control-by-control audit is in `docs/SECURITY_AUDIT.md`.

Controls that are not obvious from reading a screen

- **Screens holding PHI are wrapped in** `ProtectedScreen` — Android `FLAG_SECURE`, iOS blur-on-resign. It follows *visibility*, not mount: tab branches are never disposed, so an enable-on-mount version latched the flag on forever and never released it. Adding a screen that renders PHI means adding it to the list.
- **The whole app sits inside a 15-minute inactivity timeout.** It arms off the session provider rather than starting unconditionally — an earlier version read the session only when the timer fired, caught it mid-restore, treated that as signed-out and never re-armed.
- **Clinical free-text fields disable autocorrect and suggestions.** The OS otherwise learns typed words into the personal dictionary and suggests them *in other apps* — a diagnosis leaking with no consent record and no way to revoke it.
- `debugPrint` **is not stripped from release builds.** It forwards to `print` and reaches logcat, and the lint does not catch it, so a `kDebugMode` guard on every call site is the only control.
- **Crash reports never carry PHI.** A `Failure`'s message can quote the server's detail verbatim, so only the kind and the machine code are forwarded. Breadcrumbs are route names, never arguments.
- **The audit log is server-side only**, deliberately. A client-side log is evidence the audited party can edit.

Where web is weaker, and why

CONTROL	NATIVE	WEB
Screenshot blocking	`YES`	`NO EQUIVALENT EXISTS`
Certificate pinning	`WIRED`	`NOT EXPRESSIBLE`
Credential storage	`KEYCHAIN / KEYSTORE`	`MEMORY ONLY`
Clinical cache	`ENCRYPTED`	`DISABLED`
Video consultation	`YES`	`NOT SUPPORTED`

On web, "secure storage" is `localStorage`, so the refresh token and session id are kept **in memory only** and never written. The consequence is deliberate and visible: reloading the tab signs you out. That is the right trade for a credential that unlocks health records — and especially for the console, where the same credential can suspend an account. The real fix is a same-site `HttpOnly` refresh cookie, which script cannot read at all; setting `WEB_ORIGINS` on the API turns that on.

> **Certificate pinning is wired and its pins are empty**
>
> Empty means disabled, and that is the kill switch — a pin is the one control that can permanently brick an installed fleet. Populate `AppConfig.pinsForEnvironment` before any external release, always with two pins: the live certificate and its successor. `dart run tool/check_release_config.dart prod` fails while it is unset, so this is a gate rather than a note.

Testing

SUITE	COVERS
`router_redirect_test`	The redirect function across 7 roles × 9 provider statuses
`fixture_backend_test`	Cross-feature consequences: book → appointments, upload → readable, grant → visible, prescribe → patient, reschedule → old slot freed
`domain_rules_test`	Drug lists, consent expiry, cancellation, join window, holds, verification gate, ratings
`medication_schedule_test`	The dosing parser including every interval it refuses, course windows, adherence arithmetic
`api_repository_test`	The wire contract of every API-backed repository
`widget/screens_test`	Screens mounted for real, in both locales, at 1.3× and 2× text scale
`golden/screens_golden_test`	Layout regressions in light, dark and Hindi. Excluded from CI
`notification_rules_test`	Quiet-hour wrapping, mandatory kinds that cannot be silenced
`inactivity_timeout` · `screen_protection` · `certificate_pinning` · `sensitive_clipboard`	The four security controls

> **Those four security tests exist because every control they cover fails open and silently.** A broken inactivity timer, a screenshot flag that never re-enables, a pin that is not applied, a clipboard that never clears. None of them throw, none look wrong on screen, and two of the four were already broken when the tests were written.

Dates make tests flaky by accident

The fixture seed is relative to "now". Two have already bitten: three booking tests reached for "tomorrow" and slots return nothing on a Sunday, so they failed one day in seven; and the goldens render seeded dates, which only survived a rollover because the test font draws every digit as an identical box. Use `FixtureSeed.pinClock(at)` — it returns its own undo, so a test cannot date-lock the ones after it.

The same applies to times: anything resolving a day in IST should be tested with explicit instants rather than local wall-clock times, or it answers differently on a UTC CI box than on a laptop in India.

Still untested

The operator console's screens have no widget tests, and the Firestore transactions — double-booking and refresh rotation — need the emulator.

12 · PLATFORM

What is not finished

AREA	STATE	DETAIL
Credentials backend	`~15%`	Submission and checklist only. Document upload, DigiLocker and MFA enrolment are still fixtures.
Payments	`NOT BUILT`	`BookingPolicy.requiresPayment()` returns false. Stripe is likely wrong for INR domestic — Razorpay, Cashfree or PhonePe with UPI are the practical options.
Apple token revocation	`PARTIAL`	Works only if the user signed in with Apple in that same app run. A cold-start deletion needs the server to revoke using a stored refresh token. Required by App Store review.
Google Sign-In on iOS	`NEEDS SETUP`	Wants `CFBundleURLTypes` with the reversed client id. The block sits commented out in `Info.plist`. Without it the consent sheet opens and never returns.
Certificate pins	`EMPTY`	Disabled until populated. A release gate fails while they are unset.
Web auth persistence	`BY DESIGN`	A tab reload signs you out until `WEB_ORIGINS` turns on cookie auth.
Console widget tests	`NONE`	Its redirect logic is unit-tested; its screens are not.

Before launch

Beyond the code: a production Firebase project, Play and Apple developer accounts, production 100ms and Twilio credentials, NMC provider verification, DPDP compliance artefacts, a payment gateway decision, and the deployment steps that cannot be done from a repository — populating the pins, setting `WEB_ORIGINS`, locking bucket retention on the prescriptions prefix, and uploading the APNs key to Firebase.

The release AAB is around 80 MB because of the WebRTC natives. Ship the App Bundle, never the universal APK.

13 · REFERENCE

Reference tables

Roles

ROLE	WHERE THEY WORK
`patient`	Mobile app, patient shell
`provider`	Mobile app, provider shell — only when approved
`supervisor`	Console — review, approve, reject, moderate
`supportL1 · supportL2`	Console — tickets
`admin`	Console — everything
`unassigned`	Nowhere. Routed to the blocked screen

Statuses

KIND	VALUES
Account	`active · pending · suspended · deactivated`
Provider	`draft · submitted · underReview · approved · rejected · resubmitRequested · suspended · deactivated · notApplicable`
Prescription	`DRAFT · ISSUED · CANCELLED · SUPERSEDED`

A non-active account goes to a terminal **blocked** screen with an explanation, never a silent sign-out. Only `approved` opens the provider shell.

Scopes

Patient — `profile:read/write`, `doctor:search`, `appointment:create/cancel`, `records:read_own/write_own`, `consent:grant/revoke/view_log`, `prescription:read_own`, `consultation:join`, `rating:write`, `support:ticket_create`.

Approved provider — `profile:read/write_limited`, `availability:write`, `appointments:read_own`, `records:read_granted/request_access`, `prescription:write`, `consultation:host/join`, `ratings:read_own`.

An unapproved provider holds exactly two: `profile:read` and `credentials:submit`. That is the whole reason verification is a *status* rather than a second role — the scope set collapses to almost nothing until a supervisor approves, and there is no path by which an unapproved doctor can see a patient record or issue a prescription.

The server matrix is the authority. The Flutter fixture mirrors it so fixture-backed screens gate identically, but a client copy of a permission table is a convenience for the UI, never a control: every endpoint re-checks.

Rules that are easy to break

1. Never add a package before the code that imports it.
2. Never `print()`. A stray print leaks PHI into logcat.
3. Clinical data on the device goes through `ClinicalCache`, or not at all.
4. Never throw from a release build-type block in Gradle.
5. Release builds fail closed without `android/key.properties`. That is deliberate.
6. Write Riverpod providers by hand; the generator does not work here.
7. Use `flutter pub add`, not hand-pinned versions.
8. Run `dart format lib test` before committing. CI enforces it.
9. `functions/` sources are CRLF; `lib/` is LF.
10. A new Firestore query usually needs a new composite index.
11. Never add a scope without an endpoint behind it.
12. Crash reports must never carry PHI.
13. No clinical content in a notification title or body.
14. The consent gate renders from `TelemedicineConsent`; changing the wording means bumping the version.

MiDoctor — Flutter client and the MiDoctor API. Source of record for anything that contradicts this page: `CLAUDE.md`, `functions/README.md` and `docs/SECURITY_AUDIT.md`.